

Lab 9: Recursion

Due **Sunday, November 19, 2022 at 8:00PM ET** on Gradescope.

Each lab is graded for attendance (individual) and completion (group).

Note: Don't forget to add your pod members to the Gradescope submission!

Getting started

1. One of your group members should create an editable copy of this worksheet to fill out and share the document with all other group members. (*File* → *Make a Copy* in the menu.)
2. Fill in the table below based on the letters given to each group member. If your group does not have exactly four members, please work out a strategy for sharing the roles that works for you.

	Name	Role (rotates each week)
A	Shikha Nukala	Scribe
B	Sonya Konon	Coder I
C		Coder II
D		Leader

3. The **scribe** should do a find/replace (select *Edit > Find and replace* from the menu) to change all instances of "Shikha" to that member's name, "Sonya" to that member's name, and so on.

Formatting

Please do not change the formatting of the document or which exercises appear on which slides. We use the placement of questions on each page to assist with grading.

Questions?

You are expected to lead the meeting yourselves and collaboratively work through the exercises, but please let us know if you have questions! We're here if you need us and will be checking in throughout the lab.

Note

These lab exercises were developed for in-Shikhand virtual labs. Depending on the way you and your group are working, you are welcome to interpret the exercise instructions however works best (e.g. "share screen" means something different if you're in-person vs. virtual).

Warm-Up

Fill in the following table with true or false to refresh your memory on recursion!

Discussion Leader	Characteristic	Linear	Tail	Tree
Shikha	Uses many stack frames			X
Sonya	Function calls itself for more than 1 time	X		
Person C	Terminates on a base case		X	

Recursion Types and Relationships

Answer the following questions in the table below:

Discussion Leader	Question	Answer
Person D	What prevents something from being tail recursive?	If we have to go backwards in the recursion where the operation has to be done after the function call it cannot be tail recursive.
Shikha	Why is tail recursion “better” than general recursion?	It is better because it saves code space as after the recursive call there is no need to add more code.
Sonya	Is tail recursion or iteration more costly in terms of memory?	Tail recursion is more costly in memory and complexity because loops are less costly than recursive calls.

Printing a Linked List Recursively

Recursive algorithms can operate on data structures like linked lists. In this section, you will write code to print a linked list using recursion. This problem will be using a singly-linked list with its structure shown below:

```
struct Node {
    Node *next;
    int datum;
};

class SinglyIntList {
    Node *first; // points to first Node in list, or nullptr if list is
empty

    // assume constructors, etc. are all implemented

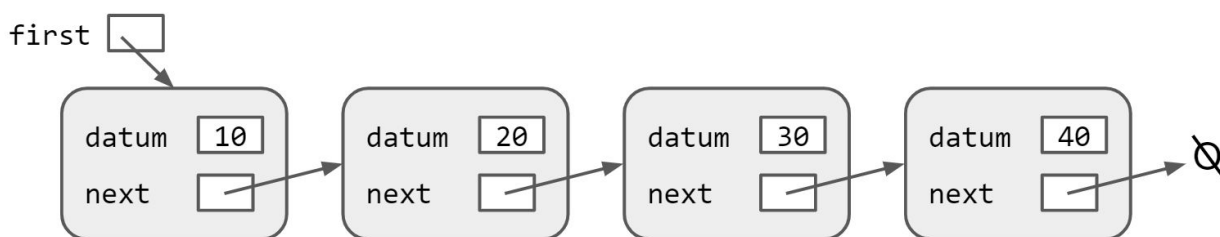
    void print_helper(){ // helper to call recursive print function
        print_list(first);
    }

    void print_list(Node* n);
};
```

We will develop a **recursive** function `print_list` within a class `SinglyIntList` (non-templated singly linked list) to do so, in conjunction with the helper.

```
void SinglyIntList::print_list();
```

Consider a `SinglyIntList` list with the following elements:



Running `list.print_list()` would print 10 20 30 40.

To review from last lab and warm up for this exercise, answer the below questions:

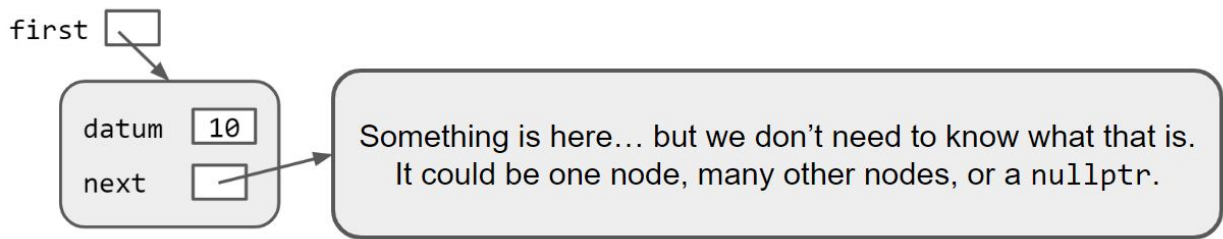
Question	Discussion Leader	Your Answer
Write the code for traversing and printing a linked list using a loop.	Sonya	<pre>void SinglyIntList::print_list_loop(Node *n) { while(n != NULL) cout << n->datum << endl; n++; } }</pre>
How do we know we've reached the end of the list in the loop?	Person C	We know we have reached the end of the list when there is a nullptr. Then there is no value at the pointer so we return without doing anything.

Every recursive algorithm needs A) base case(s), and B) recursive case(s). The base case for our algorithm will be related to the structure of the linked list we are operating on.

Question	Discussion Leader	Your Answer
The base case for a recursive algorithm is typically the simplest or smallest input to an algorithm. What is the simplest or smallest linked list and what should you do in this case?	Person D	The simplest linked list is the list of nullptrs and in that case we cannot print anything so we add a base case for finding nullptrs and return the function without any recursive calls if this is the case.

For the recursive case, we need a way to turn a larger problem into a smaller one. In the case of a linked list, this will be turning a longer list into a shorter one. We will need to (1) process one node, (2) make a recursive call on the list, minus the current node.

The following diagram shows how we can process one node (with datum 10), then use the recursive step to process the remainder of the list.



Question	Discussion Leader	Your Answer
What work does print_list need to do on each node?	Shikha	It needs to print the datum value at each node then increment the node to the next node to print that one.
What should you pass in to the recursive call to print the nodes after the current node?	Sonya	We need to pass in the next node to the current node i.e. node->next. Print until we are at the end of the list.

Based on these answers, implement the print_list function below (tail recursively).

Discussion Leader	Your Answer
Person C	<pre>void print_helper(){ print_list(first); } void SinglyIntList::print_list(Node* n) { cout << n->datum << endl; if(n->next == NULL) { return; } print_list(n->next); }</pre>

Feels Like We Only Go Backwards

We will now extend this code to print the list in reverse order (from the back of the list to the front of the list)

```
void print_list_backwards();
```

In the previous exercise, you analyzed the execution of `print_list`. The stack frames build up in one direction and return in the other direction.

Consider these questions to help get started with the implementation of this problem.

Question	Discussion Leader	Your Answer
Based on the previous question, where would you put the <code>cout</code> statement so the elements are printed only after the recursive function call returns?	Person D	We would have to put the print statements before the recursive statements but the recursive statements have to be different depending on the base case.

Look at your code for `print_list` and consider which parts of the code you will be able to reuse. Keeping this in mind, implement the `print_list_backwards` function (doesn't have to be tail recursive).

Discussion Leader	Your Implementation
Shikha	<pre>void print_backwards_helper() { print_list_backwards(first); } void SinglyIntList::print_list_backwards(Node* n) { if(n == NULL){ cout << n->prev->datum << endl; n->prev->datum == NULL; print_list_backwards(n->prev); } print_list_backwards(n->next); }</pre>

Sum Implementation

Using the recurrence relation from the previous exercise, write code for the recursive `sum()` function using the Node structure below.

```
struct Node {  
    int datum;  
    Node *next;  
  
    Node(int datum, Node *next) : datum(datum), next(next) {}  
};
```

```
// EFFECTS: Returns the sum of the elements in the list.
```

```
int sum(Node *node) {
```

```
    int current = node->datum;  
    return current + sum(node->next);
```

```
}
```

```
...
```

```
};
```

Tail Recursion Identification

Which of the following are tail recursive? Indicate in the box below each code snippet.

Sonya	Person C	Person D
<pre>//REQUIRES: x >= 0 int foo(int x) { if (!x) { return 42; } return foo(foo(x - 1)); }</pre>	<pre>//REQUIRES: x >= 0 int foo(int x) { if (!x) { return 17; } return foo(x - 1); }</pre>	<pre>//REQUIRES: x >= 0 int foo(int x) { if (!x) { return 5; } return 1 + foo(x - 1); }</pre>
This is tail recursive because the last operation carried out by the function is the call to foo.	This is tail recursive because the last function call is to the function foo.	This is not tail recursive because the last operation is the addition of one to the foo call.

Tail Recursion Sequencing

Implement the following function using **tail** recursion:

```
// EFFECTS: checks if elt is in the given sequence
// Examples: (1, []) -> False; (1, [2, 1]) -> True; (4, [2, 1]) ->
False
```

```
template <typename IterType>
bool isInSequence(IterType begin, IterType end, int elt) {
```

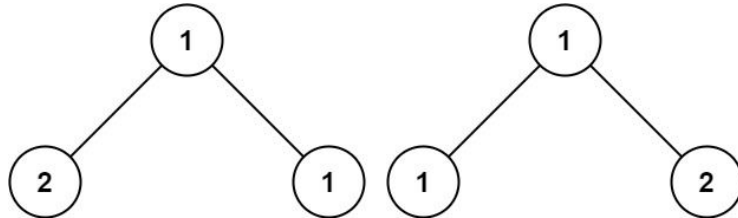
```
    IterType current = begin;
    if(*current == elt){
        return true;
    }
    if(*current == *end){
        return false;
    }
    isInSequence(current + 1; end, elt);
```

```
}
```

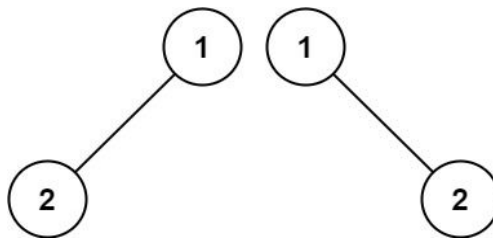

Tree Recursion

You are decorating your garden and saw an awesome tree on pinterest! You want to buy that same exact tree from your local nursery, but don't want to spend the time browsing the entire website to find the same exact tree. Using your EECS 280 skills, you decide to write a function to determine if two trees are identical!

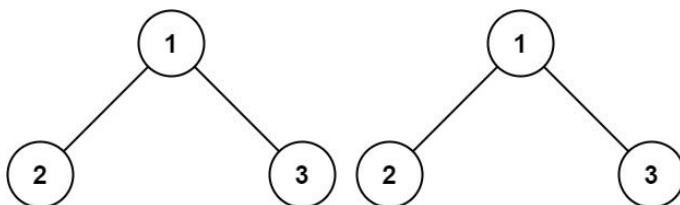
Examples:



False; the branches off the root are different



True only one branch on the root



True the same branches for the same root

Note: We are using the Node struct definition from Project 5. Each Node represents one element in the tree. The left and right pointers are "branches" to connect other Nodes.

```
template <typename T>
struct Node {
    T datum;
    Node *left;
    Node *right;
};
```

// REQUIRES: root_one points to a valid tree and root_two points to a valid tree. Note that either trees may be empty.

// EFFECTS: Returns true if the tree rooted at root_one and the tree rooted at root_two are identical.

```
bool isSameTree(Node* root_one, Node* root_two) {
```

```
// base case(s)
// check if the node pointers are null and if they are then return false
if((root_one == NULL || root_two == NULL) && (root_one != root_two))
{
    return false;
}

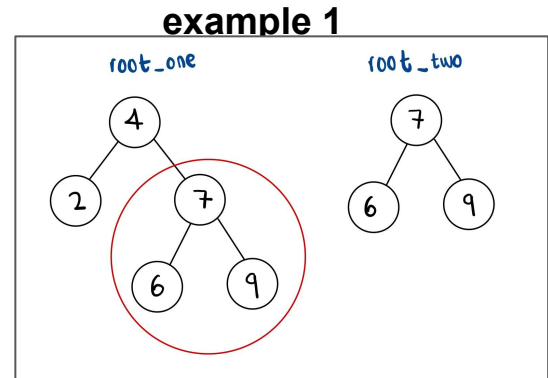
// recursive call(s)
if(root_one == root_two)
{
    return isSameTree(root_one->next, root_two->next);
}
```

```
}
```

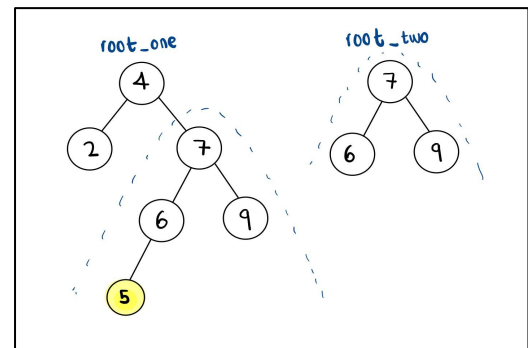
Instead of seeing if root_two is the same tree as root_one, let's see if root_two is a **subtree** of root_one.

In **example 1**, the tree rooted at root_two is said to be a subtree of the tree rooted at root_one.

This is because the tree rooted at root_one has a subtree (circled in red) that has the same structure and node values of the tree rooted at root_two



However, in **example 2**, the tree rooted at root_two is not a subtree of root_one because of the extra node with value 10 (highlighted in yellow) that makes the structures dissimilar



Hint : take advantage of isSameTree

// REQUIRES: root_one points to a tree and root_two points to another tree. Note that either trees may be empty.

// EFFECTS: Returns true if the tree rooted at root_two is a subtree of the tree rooted at root_one

```
bool isSubtree(Node* root_one, Node* root_two) {
```

```
    // base case(s)
    // check if the node pointers are null and if they are then return false
    if(root_two == NULL || root_one == NULL)
    {
        return false;
    }

    // recursive call(s)
    if(root_one == root_two)
    {
        return isSameTree(root_one->next, root_two->next);
    }
```

```
}
```

Submit worksheet

The team is responsible for submitting to Gradescope by the due date listed on the first page. We strongly recommend submitting your worksheet the same day that you meet as a pod!

The Scribe is responsible for making the final submission to Gradescope. You need to add your other pod members to the submission after it is uploaded using this link:

Lab 1 Worksheet

● **UNGRADED**

GROUP
John
 View or edit group

TOTAL POINTS
- / 1 pts